# Socket Programming

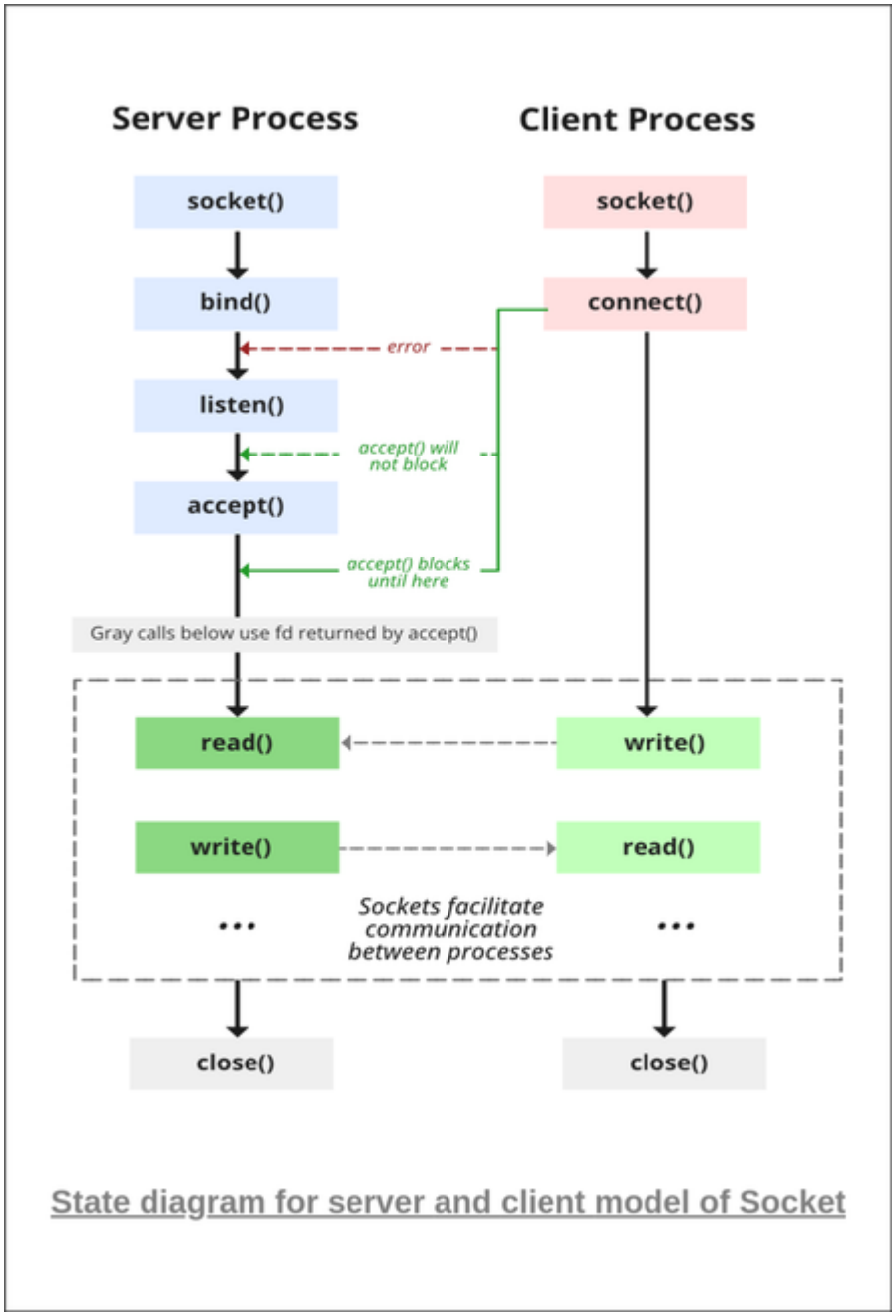
**Computer Networks**

**Course Code: BCSE308P**
**Faculty:Anusha R**

**What is Socket Programming?**

Socket programming is a way of connecting two nodes on a network to communicate with each other. One socket(node) listens on a particular port at an IP, while the other socket reaches out to the other to form a connection. The server forms the listener socket while the client reaches out to the server.

**State Diagram for Server and Client Model**




*State diagram for server and client model of Socket*

**Stages for Server**

# Socket Programming

**Computer Networks**

**Course Code: BCSE308P**
**Faculty:Anusha R**

The server is created using the following steps:

**1. Socket Creation**

int sockfd = socket(domain, type, protocol)

- **sockfd:** socket descriptor, an integer (like a file handle)
- **domain:** integer, specifies communication domain. We use AF_ LOCAL as defined in the POSIX standard for communication between processes on the same host. For communicating between processes on different hosts connected by IPV4, we use AF_INET and AF_I NET 6 for processes connected by IPV6.
- **type:** communication type
  SOCK_STREAM: TCP(reliable, connection-oriented)
  SOCK_DGRAM: UDP(unreliable, connectionless)
- **protocol:** Protocol value for Internet Protocol(IP), which is 0. This is the same number that appears on the protocol field in the IP header of a packet.(man protocols for more details)

**2. Setsockopt**

This helps in manipulating options for the socket referred by the file descriptor sockfd. This is completely optional, but it helps in reuse of address and port. Prevents error such as: “address already in use”.

int setsockopt(int sockfd, int level, int optname,  const void *optval, socklen_t optlen);

**3. Bind**

int bind(int sockfd, const struct sockaddr *addr, socklen_t addrlen);

After the creation of the socket, the bind function binds the socket to the address and port number specified in addr(custom data structure). In the example code, we bind the server to the localhost, hence we use INADDR_ANY to specify the IP address.

**4. Listen**

int listen(int sockfd, int backlog);

It puts the server socket in a passive mode, where it waits for the client to approach the server to make a connection. The backlog, defines the maximum length to which the queue of pending connections for sockfd may grow. If a connection request arrives when the queue is full, the client may receive an error with an indication of ECONNREFUSED.

**5. Accept**

int new_socket= accept(int sockfd, struct sockaddr *addr, socklen_t *addrlen);

It extracts the first connection request on the queue of pending connections for the listening socket, sockfd, creates a new connected socket, and returns a new file descriptor referring to

that socket. At this point, the connection is established between client and server, and they are ready to transfer data.

**Stages for Client**

**1. Socket connection:** Exactly the same as that of server's socket creation

**2. Connect:** The connect() system call connects the socket referred to by the file descriptor sockfd to the address specified by addr. Server's address and port is specified in addr.

int connect(int sockfd, const struct sockaddr *addr, socklen_t addrlen);

**UDP**

UDP is a connectionless protocol. There is no connection is established between the client and server. Creating a Standard UDP Client/Server is discussed here

In UDP, the client does not form a connection with the server like in TCP; instead, It just sends a datagram. Similarly, the server needs not to accept a connection and just waits for datagrams to arrive. We can call a function called connect() in UDP but it does not result in anything like it does in TCP. There is no 3-way handshake. It just checks for any immediate errors and stores the peer's IP address and port number. connect() is stores peer's addresses so no need to pass server address and server address length arguments in sendto().

```
/ server program for udp connection

#include <stdio.h>

#include <strings.h>

#include <sys/types.h>

#include <arpa/inet.h>

#include <sys/socket.h>

#include<netinet/in.h>

#define PORT 5000

#define MAXLINE 1000


// Driver code

int main()

{

   char buffer[100];
```

```
    char *message = "Hello Client";
    int listenfd, len;
    struct sockaddr_in servaddr, cliaddr;
    bzero(&servaddr, sizeof(servaddr));

    // Create a UDP Socket
    listenfd = socket(AF_INET, SOCK_DGRAM, 0);
    servaddr.sin_addr.s_addr = htonl(INADDR_ANY);
    servaddr.sin_port = htons(PORT);
    servaddr.sin_family = AF_INET;

    // bind server address to socket descriptor
    bind(listenfd, (struct sockaddr*)&servaddr, sizeof(servaddr));

    //receive the datagram
    len = sizeof(cliaddr);
    int n = recvfrom(listenfd, buffer, sizeof(buffer),
          0, (struct sockaddr*)&cliaddr,&len); //receive message from server
    buffer[n] = '\0';
    puts(buffer);

    // send the response
    sendto(listenfd, message, MAXLINE, 0,
         (struct sockaddr*)&cliaddr, sizeof(cliaddr));
}
```

UDP Client code :

```
// udp client driver program
```

# Socket Programming

**Computer Networks**

**Course Code: BCSE308P**
**Faculty:Anusha R**

```
#include <stdio.h>
#include <strings.h>
#include <sys/types.h>
#include <arpa/inet.h>
#include <sys/socket.h>
#include<netinet/in.h>
#include<unistd.h>
#include<stdlib.h>

#define PORT 5000
#define MAXLINE 1000

// Driver code
int main()
{
    char buffer[100];
    char *message = "Hello Server";
    int sockfd, n;
    struct sockaddr_in servaddr;

    // clear servaddr
    bzero(&servaddr, sizeof(servaddr));
    servaddr.sin_addr.s_addr = inet_addr("127.0.0.1");
    servaddr.sin_port = htons(PORT);
    servaddr.sin_family = AF_INET;

    // create datagram socket
    sockfd = socket(AF_INET, SOCK_DGRAM, 0);
```

# Socket Programming

**Computer Networks**

**Course Code: BCSE308P**
**Faculty:Anusha R**

```
    // connect to server
    if(connect(sockfd, (struct sockaddr *)&servaddr, sizeof(servaddr)) < 0)
    {
        printf("\n Error : Connect Failed \n");
        exit(0);
    }

    // request to send datagram
    // no need to specify server address in sendto
    // connect stores the peers IP and port
    sendto(sockfd, message, MAXLINE, 0, (struct sockaddr*)NULL, sizeof(servaddr));

    // waiting for response
    recvfrom(sockfd, buffer, sizeof(buffer), 0, (struct sockaddr*)NULL, NULL);
    puts(buffer);
    // close the descriptor
    close(sockfd);
}
```

```
shubham@gfg-desktop:~$ gcc server.c
shubham@gfg-desktop:~$ gcc -o server server.c
shubham@gfg-desktop:~$ ./server
Hello Server
shubham@gfg-desktop:~$
```

```
shubham@gfg-desktop:~$ gcc -o client clientUDP.c
shubham@gfg-desktop:~$ ./client
Hello Client
shubham@gfg-desktop:~$
```

**Server-side Code in C (TCP)**

```
// Server side C program to demonstrate Socket programming
```

# Socket Programming

**Computer Networks**

**Course Code: BCSE308P**
**Faculty:Anusha R**

```
#include <unistd.h>
#include <stdio.h>
#include <sys/socket.h>
#include <stdlib.h>
#include <netinet/in.h>
#include <string.h>
#define PORT 8080

int main(int argc, char const *argv[]) {
    int server_fd, new_socket, valread;
    struct sockaddr_in address;
    int opt = 1;
    int addrlen = sizeof(address);
    char buffer[1024] = {0};
    char *hello = "Hello from server";

    // Creating socket file descriptor
    if ((server_fd = socket(AF_INET, SOCK_STREAM, 0)) == 0) {
        perror("socket failed");
        exit(EXIT_FAILURE);
    }

    // Forcefully attaching socket to the port 8080
    if (setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR | SO_REUSEPORT, &opt,
sizeof(opt))) {
        perror("setsockopt");
        exit(EXIT_FAILURE);
    }
    address.sin_family = AF_INET;
```

```
    address.sin_addr.s_addr = INADDR_ANY;
    address.sin_port = htons(PORT);


    // Forcefully attaching socket to the port 8080
    if (bind(server_fd, (struct sockaddr *)&address, sizeof(address)) < 0) {
        perror("bind failed");
        exit(EXIT_FAILURE);
    }
    if (listen(server_fd, 3) < 0) {
        perror("listen");
        exit(EXIT_FAILURE);
    }
    if ((new_socket = accept(server_fd, (struct sockaddr *)&address, (socklen_t*)&addrlen)) <
0) {
        perror("accept");
        exit(EXIT_FAILURE);
    }
    valread = read(new_socket, buffer, 1024);
    printf("Message from client: %s\n", buffer);
    send(new_socket, hello, strlen(hello), 0);
    printf("Hello message sent\n");
    return 0;
}
// Client side C program to demonstrate Socket programming
#include <stdio.h>
#include <sys/socket.h>
#include <arpa/inet.h>
#include <unistd.h>
#include <string.h>
```

# Socket Programming

**Computer Networks**

**Course Code: BCSE308P**
**Faculty:Anusha R**

```
#define PORT 8080

int main(int argc, char const *argv[]) {
    int sock = 0, valread;
    struct sockaddr_in serv_addr;
    char *hello = "Hello from client";
    char buffer[1024] = {0};

    // Creating socket file descriptor
    if ((sock = socket(AF_INET, SOCK_STREAM, 0)) < 0) {
        printf("\n Socket creation error \n");
        return -1;
    }

    serv_addr.sin_family = AF_INET;
    serv_addr.sin_port = htons(PORT);

    // Convert IPv4 and IPv6 addresses from text to binary form
    if (inet_pton(AF_INET, "127.0.0.1", &serv_addr.sin_addr) <= 0) {
        printf("\nInvalid address/ Address not supported \n");
        return -1;
    }

    if (connect(sock, (struct sockaddr *)&serv_addr, sizeof(serv_addr)) < 0) {
        printf("\nConnection Failed \n");
        return -1;
    }
    send(sock, hello, strlen(hello), 0);
    printf("Hello message sent\n");
```

```
    valread = read(sock, buffer, 1024);

    printf("Message from server: %s\n", buffer);

    return 0;

}
```

Here is a basic example of socket programming in C for both the server and client side using TCP in a network.

**Server-side Code in C**

```
// Server side C program to demonstrate Socket programming
#include <unistd.h>
#include <stdio.h>
#include <sys/socket.h>
#include <stdlib.h>
#include <netinet/in.h>
#include <string.h>
#define PORT 8080

int main(int argc, char const *argv[]) {
    int server_fd, new_socket, valread;
    struct sockaddr_in address;
    int opt = 1;
    int addrlen = sizeof(address);
    char buffer[1024] = {0};
    char *hello = "Hello from server";

    // Creating socket file descriptor
    if ((server_fd = socket(AF_INET, SOCK_STREAM, 0)) == 0) {
        perror("socket failed");
        exit(EXIT_FAILURE);
    }

    // Forcefully attaching socket to the port 8080
    if (setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR | SO_REUSEPORT, &opt,
sizeof(opt))) {
        perror("setsockopt");
        exit(EXIT_FAILURE);
    }
    address.sin_family = AF_INET;
    address.sin_addr.s_addr = INADDR_ANY;
    address.sin_port = htons(PORT);

    // Forcefully attaching socket to the port 8080
    if (bind(server_fd, (struct sockaddr *)&address, sizeof(address)) < 0) {
```

```
        perror("bind failed");
        exit(EXIT_FAILURE);
    }
    if (listen(server_fd, 3) < 0) {
        perror("listen");
        exit(EXIT_FAILURE);
    }
    if ((new_socket = accept(server_fd, (struct sockaddr *)&address, (socklen_t*)&addrlen)) <
0) {
        perror("accept");
        exit(EXIT_FAILURE);
    }
    valread = read(new_socket, buffer, 1024);
    printf("Message from client: %s\n", buffer);
    send(new_socket, hello, strlen(hello), 0);
    printf("Hello message sent\n");
    return 0;
}
```

**Client-side Code in C**

```
// Client side C program to demonstrate Socket programming
#include <stdio.h>
#include <sys/socket.h>
#include <arpa/inet.h>
#include <unistd.h>
#include <string.h>
#define PORT 8080

int main(int argc, char const *argv[]) {
    int sock = 0, valread;
    struct sockaddr_in serv_addr;
    char *hello = "Hello from client";
    char buffer[1024] = {0};

    // Creating socket file descriptor
    if ((sock = socket(AF_INET, SOCK_STREAM, 0)) < 0) {
        printf("\n Socket creation error \n");
        return -1;
    }

    serv_addr.sin_family = AF_INET;
    serv_addr.sin_port = htons(PORT);

    // Convert IPv4 and IPv6 addresses from text to binary form
    if (inet_pton(AF_INET, "127.0.0.1", &serv_addr.sin_addr) <= 0) {
        printf("\nInvalid address/ Address not supported \n");
```

```
    return -1;
  }

  if (connect(sock, (struct sockaddr *)&serv_addr, sizeof(serv_addr)) < 0) {
    printf("\nConnection Failed \n");
    return -1;
  }
  send(sock, hello, strlen(hello), 0);
  printf("Hello message sent\n");
  valread = read(sock, buffer, 1024);
  printf("Message from server: %s\n", buffer);
  return 0;
}
```

**Explanation:**

1. **Server Side**:
   - A socket is created and bound to a specific port (8080 in this case).
   - The server listens for incoming connections and accepts them.
   - After accepting the connection, it reads a message from the client and responds back.
2. **Client Side**:
   - A socket is created to connect to the server.
   - After establishing the connection, the client sends a message to the server.
   - The client then reads the response from the server and prints it.

**How to Run the Code:**

1. Compile the server and client programs separately:

   ```
   gcc server.c -o server
   gcc client.c -o client
   ```

2. Run the server program:

   ```
   ./server
   ```

3. In another terminal, run the client program:

   ```
   ./client
   ```

**Example Input/Output:**

**Client Output:**

# Socket Programming

**Computer Networks**

**Course Code: BCSE308P**
**Faculty:Anusha R**

Hello message sent
Message from server: Hello from server

**Server Output:**

Message from client: Hello from client
Hello message sent

This basic program demonstrates the communication between a client and a server using sockets in C.

In socket programming, various operations can be performed depending on the type of communication protocol (TCP/UDP) or the specific use case. Here are some basic operations or programs that can be implemented with socket programming:

**1. TCP Echo Server and Client**

- An echo server sends back whatever it receives from the client. This helps in understanding full-duplex communication between server and client.

**Server**:

- Listens for a connection and, upon receiving a message from a client, sends the same message back.

**Client**:

- Sends a message to the server, then receives and displays the echoed message.

**Use Case**: Useful for testing network and socket functionality.

**2. File Transfer Using TCP**

- A simple program to transfer files between a server and a client using socket programming.

**Server**:

- Reads a file from its local disk and sends it to the client over a socket connection.

**Client**:

- Receives the file data and saves it to its own disk.

**Use Case**: Can be used for sharing files over a network.

**3. Simple Chat Application (TCP or UDP)**

- This allows two users (client and server) to communicate with each other over a network.

**Server**:

- Listens for messages from the client, then sends a reply.

**Client**:

- Sends messages to the server and receives responses.

**Use Case**: It's the basic building block for chat applications.

**4. Broadcast Communication (UDP)**

- A program where a server sends a broadcast message to all clients within a local network.

**Server**:

- Uses UDP to broadcast a message to all clients within the same network.

**Client**:

- Listens for broadcast messages and displays them.

**Use Case**: Used for notification systems, or multicast networks for distributing data to multiple recipients.

**5. UDP Communication**

- UDP is connectionless, so it doesn't establish a dedicated connection before sending data. A basic UDP program involves sending and receiving messages using datagrams.

**Server**:

- Receives data from multiple clients without establishing a connection.

**Client**:

- Sends data to the server without maintaining a persistent connection.

**Use Case**: Useful in applications where low-latency communication is required (e.g., real-time video streaming, gaming).

**6. Multithreaded Server (TCP)**

- A server that handles multiple clients concurrently using threads or processes.

**Server**:

- Creates a new thread to handle each client's requests concurrently.

**Client**:

- Interacts with the server as usual.

**Use Case**: Used in web servers, where multiple clients may connect and send requests simultaneously.

**7. Simple HTTP Web Server**

- A very basic HTTP server that handles GET requests.

**Server**:

- Reads HTTP requests, serves static files, or sends a basic HTML response.

**Client**:

- Any web browser or a basic HTTP client that sends a request to the server.

**Use Case**: This can serve static web pages like a basic web server.

**8. Ping Utility (ICMP using Raw Sockets)**

- Implement a simple version of the "ping" command using raw sockets to send ICMP Echo Request packets and receive ICMP Echo Reply packets.

**Server**: Not required; raw sockets are used to communicate directly with the network layer.

**Client**: Sends ICMP packets to a target host and receives responses.

**Use Case**: Used to check the reachability of a host on a network.

**9. Client/Server for Remote Command Execution (TCP)**

- A server that receives commands from a client, executes them, and sends the output back to the client.

**Server**:

- Listens for commands, executes them (e.g., on the local shell), and sends the output back.

# Socket Programming

**Computer Networks**

**Course Code: BCSE308P**
**Faculty:Anusha R**

**Client**:

- Sends commands to the server and displays the server's output.

**Use Case**: Remote system administration or command execution over a network.

**10. Load Balancer (TCP)**

- A load balancer that sits between clients and multiple servers, distributing incoming requests across available servers.

**Server** (Load Balancer):

- Receives requests and forwards them to one of several backend servers.

**Client**:

- Sends requests to the load balancer, unaware of which server is handling the request.

**Use Case**: Used in web infrastructure to distribute load across servers.

1. **TCP Echo Server and Client**

```
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

#define PORT 8080

int main() {
    int server_fd, new_socket;
    struct sockaddr_in address;
    int addrlen = sizeof(address);
    char buffer[1024] = {0};

    server_fd = socket(AF_INET, SOCK_STREAM, 0);

    address.sin_family = AF_INET;
    address.sin_addr.s_addr = INADDR_ANY;
    address.sin_port = htons(PORT);

    bind(server_fd, (struct sockaddr *)&address, sizeof(address));

    listen(server_fd, 3);
```

```
    new_socket = accept(server_fd, (struct sockaddr *)&address,
(socklen_t*)&addrlen);

    while (1) {
        memset(buffer, 0, sizeof(buffer));
        read(new_socket, buffer, 1024);
        printf("Client: %s", buffer);
        send(new_socket, buffer, strlen(buffer), 0);
    }
    close(new_socket);
    return 0;
```

Client

```
}#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

#define PORT 8080

int main() {
    int sock;
    struct sockaddr_in serv_addr;
    char buffer[1024] = {0};
    char message[1024];

    sock = socket(AF_INET, SOCK_STREAM, 0);

    serv_addr.sin_family = AF_INET;
    serv_addr.sin_port = htons(PORT);
    inet_pton(AF_INET, "127.0.0.1", &serv_addr.sin_addr);

    connect(sock, (struct sockaddr *)&serv_addr, sizeof(serv_addr));

    while (1) {
        printf("Enter message: ");
        fgets(message, 1024, stdin);
        send(sock, message, strlen(message), 0);
        read(sock, buffer, 1024);
        printf("Echo from server: %s", buffer);
    }
    close(sock);
```

```
    return 0;
}
```

**Output**:

- Client sends: "Hello from client"
- Server responds: "Hello from client"

**2. File Transfer Using TCP**

**Server (File Transfer)**:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

#define PORT 8080

int main() {
    int server_fd, new_socket;
    struct sockaddr_in address;
    int addrlen = sizeof(address);
    char buffer[1024] = {0};
    FILE *fp;

    server_fd = socket(AF_INET, SOCK_STREAM, 0);
    address.sin_family = AF_INET;
    address.sin_addr.s_addr = INADDR_ANY;
    address.sin_port = htons(PORT);

    bind(server_fd, (struct sockaddr *)&address, sizeof(address));
    listen(server_fd, 3);
    new_socket = accept(server_fd, (struct sockaddr *)&address,
(socklen_t*)&addrlen);

    fp = fopen("server_file.txt", "r");
    while (fgets(buffer, 1024, fp)) {
        send(new_socket, buffer, strlen(buffer), 0);
    }
    fclose(fp);
    close(new_socket);
    return 0;
```

```
}
```

**Client (File Transfer)**:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

#define PORT 8080

int main() {
    int sock;
    struct sockaddr_in serv_addr;
    char buffer[1024] = {0};
    FILE *fp;

    sock = socket(AF_INET, SOCK_STREAM, 0);
    serv_addr.sin_family = AF_INET;
    serv_addr.sin_port = htons(PORT);
    inet_pton(AF_INET, "127.0.0.1", &serv_addr.sin_addr);

    connect(sock, (struct sockaddr *)&serv_addr, sizeof(serv_addr));
    fp = fopen("received_file.txt", "w");

    while (read(sock, buffer, 1024)) {
        fprintf(fp, "%s", buffer);
    }
    fclose(fp);
    close(sock);
    return 0;
}
```

**Output**:

- Server sends a file, server_file.txt to the client.
- Client receives and saves it as received_file.txt.

**3. Simple Chat Application (TCP)**

**Server (Chat)**:

```
#include <stdio.h>
#include <string.h>
#include <unistd.h>
```

# Socket Programming

**Computer Networks**

**Course Code: BCSE308P**
**Faculty:Anusha R**

```
#include <arpa/inet.h>

#define PORT 8080

int main() {
   int server_fd, new_socket;
   struct sockaddr_in address;
   int addrlen = sizeof(address);
   char buffer[1024] = {0};

   server_fd = socket(AF_INET, SOCK_STREAM, 0);
   address.sin_family = AF_INET;
   address.sin_addr.s_addr = INADDR_ANY;
   address.sin_port = htons(PORT);

   bind(server_fd, (struct sockaddr *)&address, sizeof(address));
   listen(server_fd, 3);
   new_socket = accept(server_fd, (struct sockaddr *)&address,
(socklen_t*)&addrlen);

   while (1) {
      read(new_socket, buffer, 1024);
      printf("Client: %s", buffer);
      printf("Enter message: ");
      fgets(buffer, 1024, stdin);
      send(new_socket, buffer, strlen(buffer), 0);
   }
   return 0;
}
```

**Client (Chat)**:

```
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

#define PORT 8080

int main() {
   int sock;
   struct sockaddr_in serv_addr;
   char buffer[1024] = {0};

   sock = socket(AF_INET, SOCK_STREAM, 0);
   serv_addr.sin_family = AF_INET;
```

```
    serv_addr.sin_port = htons(PORT);
    inet_pton(AF_INET, "127.0.0.1", &serv_addr.sin_addr);

    connect(sock, (struct sockaddr *)&serv_addr, sizeof(serv_addr));

    while (1) {
        printf("Enter message: ");
        fgets(buffer, 1024, stdin);
        send(sock, buffer, strlen(buffer), 0);
        read(sock, buffer, 1024);
        printf("Server: %s", buffer);
    }
    close(sock);
    return 0;
}
```

**Output**:

- Client and server can send and receive messages in a continuous chat.

**4. Broadcast Communication (UDP)**

**Server (UDP Broadcast)**:

```
#include <stdio.h>
#include <string.h>
#include <arpa/inet.h>

#define PORT 8080

int main() {
    int sockfd;
    struct sockaddr_in broadcast_addr;
    char message[] = "Broadcast message from server";

    sockfd = socket(AF_INET, SOCK_DGRAM, 0);
    broadcast_addr.sin_family = AF_INET;
    broadcast_addr.sin_port = htons(PORT);
    broadcast_addr.sin_addr.s_addr = inet_addr("255.255.255.255");

    while (1) {
        sendto(sockfd, message, strlen(message), 0, (struct sockaddr*)&broadcast_addr,
sizeof(broadcast_addr));
        sleep(1);
```

```
    }
    close(sockfd);
    return 0;
}
```

**Client (UDP Broadcast Listener)**:

```
#include <stdio.h>
#include <string.h>
#include <arpa/inet.h>

#define PORT 8080

int main() {
    int sockfd;
    struct sockaddr_in listen_addr;
    char buffer[1024] = {0};
    socklen_t addr_len = sizeof(listen_addr);

    sockfd = socket(AF_INET, SOCK_DGRAM, 0);
    listen_addr.sin_family = AF_INET;
    listen_addr.sin_port = htons(PORT);
    listen_addr.sin_addr.s_addr = INADDR_ANY;

    bind(sockfd, (struct sockaddr*)&listen_addr, sizeof(listen_addr));

    while (1) {
        recvfrom(sockfd, buffer, 1024, 0, (struct sockaddr*)&listen_addr, &addr_len);
        printf("Received: %s\n", buffer);
    }
    close(sockfd);
    return 0;
}
```

**Output**:

- Server continuously broadcasts "Broadcast message from server".
- Clients display this message.

**5. UDP Communication**

**Server (UDP)**:

```
#include <stdio.h>
#include <string.h>
#include <arpa/inet.h>
```

# Socket Programming

**Computer Networks**

**Course Code: BCSE308P**
**Faculty:Anusha R**

```
#define PORT 8080

int main() {
    int sockfd;
    struct sockaddr_in servaddr, cliaddr;
    char buffer[1024] = {0};
    socklen_t len = sizeof(cliaddr);

    sockfd = socket(AF_INET, SOCK_DGRAM, 0);
    servaddr.sin_family = AF_INET;
    servaddr.sin_addr.s_addr = INADDR_ANY;
    servaddr.sin_port = htons(PORT);

    bind(sockfd, (struct sockaddr*)&servaddr, sizeof(servaddr));

    recvfrom(sockfd, buffer, 1024, 0, (struct sockaddr*)&cliaddr, &len);
    printf("Client: %s", buffer);
    sendto(sockfd, buffer, strlen(buffer), 0, (struct sockaddr*)&cliaddr, len);
    close(sockfd);
    return 0;
}
```

Client (UDP)

```
#include <stdio.h>
#include <string.h>
#include <arpa/inet.h>

#define PORT 8080

int main() {
    int sockfd;
    struct sockaddr_in servaddr;
    char buffer[1024] = {0};
    char message[] = "Hello from client";

    sockfd = socket(AF_INET, SOCK_DGRAM, 0);
    servaddr.sin_family = AF_INET;
    servaddr.sin_port = htons(PORT);
    servaddr.sin_addr.s_addr = inet_addr("127.0.0.1");

    sendto(sockfd, message, strlen(message), 0, (struct sockaddr*)&servaddr,
sizeof(servaddr));
    recvfrom(sockfd, buffer, 1024, 0, NULL, NULL);
    printf("Server: %s\n", buffer);
```

```
    close(sockfd);
    return 0;
}
```

**Output**:

- Client sends "Hello from client" via UDP, and the server echoes the message.

**Advanced Socket Programming Concepts:**

1. **Non-blocking Sockets**: These allow operations to be performed without waiting for completion (e.g., asynchronous I/O).
2. **Select/Poll**: Used for monitoring multiple file descriptors (sockets) to see if they have incoming data.
3. **SSL/TLS**: Secure sockets layer for encrypted communication.

Each of these operations introduces different networking paradigms and problem-solving techniques used in real-world network programming.

**1. Polling Using poll()**

The poll() function allows a server to handle multiple clients by monitoring multiple file descriptors. This is useful for applications where multiple clients may connect simultaneously, and the server needs to respond to all of them.

Here's a simple implementation of poll() for handling multiple clients:

*Polling Server (using poll()):*

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <poll.h>

#define PORT 8080
#define MAX_CLIENTS 10

int main() {
    int server_fd, new_socket, client_socket[MAX_CLIENTS], max_clients =
MAX_CLIENTS, activity;
    struct sockaddr_in address;
    struct pollfd fds[MAX_CLIENTS];
    char buffer[1024] = {0};
    socklen_t addrlen = sizeof(address);
```

# Socket Programming

**Computer Networks**

**Course Code: BCSE308P**
**Faculty:Anusha R**

```
// Initialize all client_socket[] to 0 so that they are not checked
for (int i = 0; i < max_clients; i++) {
    client_socket[i] = 0;
}

// Create server socket
server_fd = socket(AF_INET, SOCK_STREAM, 0);
address.sin_family = AF_INET;
address.sin_addr.s_addr = INADDR_ANY;
address.sin_port = htons(PORT);

// Bind the socket
bind(server_fd, (struct sockaddr*)&address, sizeof(address));

// Listen for connections
listen(server_fd, 3);

// Initialize polling structure
fds[0].fd = server_fd;
fds[0].events = POLLIN; // Check for input events

while (1) {
    activity = poll(fds, max_clients, -1);

    // Check if there's a new connection request
    if (fds[0].revents & POLLIN) {
        new_socket = accept(server_fd, (struct sockaddr*)&address, (socklen_t*)&addrlen);
        printf("New connection from client\n");

        // Add new socket to the list of sockets
        for (int i = 1; i < max_clients; i++) {
            if (fds[i].fd == 0) {
                fds[i].fd = new_socket;
                fds[i].events = POLLIN;
                break;
            }
        }
    }

    // Check for IO operation on other sockets
    for (int i = 1; i < max_clients; i++) {
        if (fds[i].fd > 0 && fds[i].revents & POLLIN) {
            int valread = read(fds[i].fd, buffer, 1024);
            if (valread == 0) {
                // Client disconnected
                printf("Client disconnected\n");
```

```
                close(fds[i].fd);
                fds[i].fd = 0;
            } else {
                buffer[valread] = '\0';
                printf("Message from client: %s\n", buffer);
                send(fds[i].fd, buffer, strlen(buffer), 0);
            }
        }
    }
}

close(server_fd);
return 0;
}
```

**Explanation**:

- The server uses poll() to monitor multiple client connections.
- The server checks if a new client connects and if any client sends a message, and responds accordingly.

**Output**:

- Server can handle multiple clients concurrently, echoing back their messages.

---

**2. SSL/TLS Communication Using OpenSSL**

To handle encrypted communication, you can use **OpenSSL**. Below is a simple example of a secure SSL/TLS connection between a client and server.

**Note**: You'll need to install OpenSSL on your system and link it to your project in **Code::Blocks**.

*Server Code with SSL (Using OpenSSL)*

```
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <openssl/ssl.h>
#include <openssl/err.h>

#define PORT 8081

void initialize_ssl() {
    SSL_load_error_strings();
```

```
    OpenSSL_add_ssl_algorithms();
}

void cleanup_ssl() {
    EVP_cleanup();
}

int main() {
    int server_fd;
    struct sockaddr_in address;
    int addrlen = sizeof(address);

    // Initialize OpenSSL
    initialize_ssl();

    SSL_CTX *ctx = SSL_CTX_new(TLS_server_method());
    if (!ctx) {
        perror("Unable to create SSL context");
        ERR_print_errors_fp(stderr);
        exit(EXIT_FAILURE);
    }

    // Set the certificate and key for SSL
    SSL_CTX_use_certificate_file(ctx, "server.crt", SSL_FILETYPE_PEM);
    SSL_CTX_use_PrivateKey_file(ctx, "server.key", SSL_FILETYPE_PEM);

    server_fd = socket(AF_INET, SOCK_STREAM, 0);
    address.sin_family = AF_INET;
    address.sin_addr.s_addr = INADDR_ANY;
    address.sin_port = htons(PORT);

    bind(server_fd, (struct sockaddr*)&address, sizeof(address));
    listen(server_fd, 1);

    printf("SSL server listening on port %d...\n", PORT);
    int client_socket = accept(server_fd, (struct sockaddr*)&address, (socklen_t*)&addrlen);

    SSL *ssl = SSL_new(ctx);
    SSL_set_fd(ssl, client_socket);

    if (SSL_accept(ssl) <= 0) {
        ERR_print_errors_fp(stderr);
    } else {
        char buffer[1024] = {0};
        SSL_read(ssl, buffer, sizeof(buffer));
        printf("Client message: %s\n", buffer);
        SSL_write(ssl, "Hello from SSL server", strlen("Hello from SSL server"));
```

```
    }

    SSL_shutdown(ssl);
    SSL_free(ssl);
    close(client_socket);

    SSL_CTX_free(ctx);
    cleanup_ssl();
    return 0;
}
```

*Client Code with SSL (Using OpenSSL)*

```
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <openssl/ssl.h>
#include <openssl/err.h>

#define PORT 8081

void initialize_ssl() {
    SSL_load_error_strings();
    OpenSSL_add_ssl_algorithms();
}

void cleanup_ssl() {
    EVP_cleanup();
}

int main() {
    int sock;
    struct sockaddr_in serv_addr;

    // Initialize OpenSSL
    initialize_ssl();

    SSL_CTX *ctx = SSL_CTX_new(TLS_client_method());
    if (!ctx) {
        perror("Unable to create SSL context");
        ERR_print_errors_fp(stderr);
        exit(EXIT_FAILURE);
    }

    sock = socket(AF_INET, SOCK_STREAM, 0);
    serv_addr.sin_family = AF_INET;
    serv_addr.sin_port = htons(PORT);
    inet_pton(AF_INET, "127.0.0.1", &serv_addr.sin_addr);
```

```
    connect(sock, (struct sockaddr*)&serv_addr, sizeof(serv_addr));

    SSL *ssl = SSL_new(ctx);
    SSL_set_fd(ssl, sock);

    if (SSL_connect(ssl) <= 0) {
        ERR_print_errors_fp(stderr);
    } else {
        SSL_write(ssl, "Hello from SSL client", strlen("Hello from SSL client"));

        char buffer[1024] = {0};
        SSL_read(ssl, buffer, sizeof(buffer));
        printf("Server message: %s\n", buffer);
    }

    SSL_shutdown(ssl);
    SSL_free(ssl);
    close(sock);

    SSL_CTX_free(ctx);
    cleanup_ssl();
    return 0;
}
```

**Explanation**:

- The server and client use SSL/TLS to establish a secure connection.
- Server and client exchange certificates and establish an encrypted session.
- The server sends a message, and the client receives it securely.

**Output**:

- Encrypted communication between client and server with SSL.

To run both the **client** and **server** programs (for both the polling example and SSL/TLS example) on your local machine, you need to follow these steps:

### 1. Setting up for Polling Example

*Step 1: Compile the Server Program*

- Open **Code::Blocks**.
- Create a new project for the **server**.
- Copy the server code for polling into a new C file (server_poll.c).
- Compile the server program.

# Socket Programming

**Computer Networks**

**Course Code: BCSE308P**
**Faculty:Anusha R**

*Step 2: Run the Server Program*

- Run the server program. It will start listening for client connections.
- You won't see much output initially until the client connects.

*Step 3: Compile the Client Program*

- Create another new project for the **client**.
- Copy the client-side code for the echo client (similar to the previous TCP client example, slightly adjusted for multiple clients).
- Compile the client program.

*Step 4: Run Multiple Clients*

- You can run the client multiple times in different terminals or within Code::Blocks by creating different instances of the client.
- Each client sends messages to the server, and the server will echo the messages back.

*Example:*

In **Code::Blocks**:

1. Create one project for the server and run it.
2. Create another project for the client and run multiple instances of the client.